

# Java

## Multithreading,

```
class MyRunnable implements Runnable{

    public void run(){
        System.out.println("Runnable is running");
        for(int i=1; i<=10; i++){
            System.out.println(i);
        }
    }
}

class example{
    static int priority =10;

    int data = 30;
}

class MyThread extends Thread{
    public void run(){
        System.out.println("Thread is running");
        for(int i=1; i<=10; i++){
            System.out.println(i);
        }
    }
}

public class Multithreading {

    public static void main(String[] args) throws InterruptedException {

        // example prior = new example();

        // System.out.println(example.priority);

        MyRunnable obj1 = new MyRunnable();

        Thread obj2 = new Thread(obj1);
        obj2.setName("Runnable thread");
        System.out.println(obj2.getName());
        obj2.setPriority(Thread.MIN_PRIORITY);
        System.out.println("Thread priority for runnable "
+obj2.getPriority());
    }
}
```

```
obj2.start();
System.out.println(obj2.isAlive());
obj2.join();

MyThead obj = new MyThead();
obj.setName("Thread class");
obj.setPriority(9); // ranges 1- 10

System.out.println("Thread priority for classs " +obj.getPriority());

System.out.println(obj.getName());

obj.start(); //main thread ignore

obj.join(); // main thread waits for this thread to complete

System.out.println("Multithreading");

// process vs thread

// process -> separate execution (separate memory)

// thread ->unit of execution (shared memory)

// thread lifecycle
// new
// runnable
// running
// non-runnable
// terminated

// how to create thread

// thread class, runnable interface
```

```

    }
}

class Counter {

    int count =0;

    synchronized void increment(){
        count++;
    }
}

public class lambda {

    public static void main(String[] args) throws InterruptedException {

        System.out.println("lambda expression above");

        Counter ct = new Counter();
        // System.out.println(ct.count);

        // ct.increment();

        // System.out.println(ct.count);
        Thread obj = new Thread( ()->{
            for(int i=1; i<=30000; i++){
                ct.increment();
            }
        });

        Thread obj2 = new Thread( ()->{
            for(int i=1; i<=20000; i++){
                ct.increment();
            }
        });

        obj.start();
        obj2.start();

        obj.join();
        obj2.join();
    }
}

```

```
        System.out.println(ct.count);

        System.out.println("Lambda expression below");

    }
}
```

## Theory Notes

### 1. Creating Threads

- **Thread class** → extend `Thread` and override `run()`.
- **Runnable interface** → implement `Runnable` and pass to `Thread`.
- **Lambda expression** → shorthand for `Runnable` implementation.

### 2. Thread Lifecycle

- **New** → thread created but not started.
- **Runnable** → thread ready to run.
- **Running** → thread executing.
- **Non-runnable** → waiting/sleeping.
- **Terminated** → finished execution.

### 3. Thread Methods

- `start()` → begins execution.
- `join()` → waits for thread to finish.
- `setPriority()` → sets priority (1–10).
- `isAlive()` → checks if thread is still running.

### 4. Synchronization

- `synchronized` keyword ensures thread safety when multiple threads access shared resources.
- Prevents race conditions.

## Coding Round Questions

1. Create a thread using the `Thread` class that prints numbers from 1 to 10.
2. Create a thread using the `Runnable` interface that prints numbers from 1 to 10.
3. Demonstrate setting thread names and priorities.
4. Write a program to check if a thread is alive before and after execution.
5. Use `join()` to make the main thread wait until child threads finish.

6. Create a shared counter class and increment it using multiple threads. Show how synchronization prevents race conditions.
7. Demonstrate thread creation using a lambda expression.

## Challenge

### Problem: Banking System with Multithreading

#### Requirements:

- Create a `BankAccount` class with a private balance.
- Add synchronized methods `deposit(amount)` and `withdraw(amount)`.
- Create multiple threads simulating customers depositing and withdrawing money.
- Ensure thread safety so balance never goes negative.
- Print transaction logs with thread names.